

Лабораторная работа №1

Построение простейшего синтаксического анализатора выражений

Теоретическая часть

Лексема - минимальная единица языка, имеющая самостоятельный смысл. Например, для языка C++ лексеммами являются 'main', '38.9e-4', '{' и т.п.

Синтаксический анализатор предназначен для распознавания синтаксической структуры из представленных лексических композиций (т.е. потока лексемм в том порядке, в каком они поступают на вход синтаксического анализатора). Проще говоря (опять в качестве примера возьмём C++), синтаксический анализатор должен «проглотить» последовательность лексемм **int a = 5;** и отказаться принимать такую последовательность тех же самых лексемм: **a = 5 int;**

Ниже будет рассмотрен один из множества алгоритмов синтаксического анализа арифметических выражений, позволяющий вычислить их значение.

Вычисление значений выражений с помощью обратной польской нотации

Из большого числа методов и алгоритмов вычисления значений арифметических выражений наибольшее распространение получил метод трансляции с помощью обратной польской записи, которую предложил польский математик Я. Лукашевич. Следуя высказыванию Ньютона, что "в изучении наук примеры важнее правил", рассмотрим этот метод на некотором обобщённом примере.

Пусть задано простое арифметическое выражение вида:

$$(A+B)*(C+D)-E \quad (1)$$

Представим это выражение в виде дерева, в котором узлам соответствуют операции, а ветвям - операнды. Построение начнем с корня, в качестве которого выбирается операция, выполняющаяся последней.левой ветви соответствует левый операнд операции, а правой ветви - правый. Дерево выражения (1) показано на рис. 1.

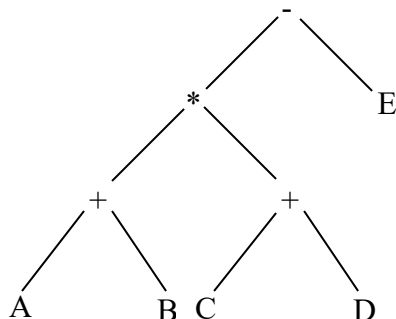


Рис. 1

Совершим обход дерева, под которым будем понимать формирование строки символов из символов узлов и ветвей дерева. Обход будем совершать от самой

левой ветви вправо и узел переписывать в выходную строку только после рассмотрения всех его ветвей. Результат обхода дерева имеет вид:

$$AB+CD+*E- \quad (2)$$

Характерные особенности выражения (2) состоят в следовании символов операций за символами операндов и в отсутствии скобок. Такая запись называется *обратной польской записью*.

Обратная польская запись обладает рядом замечательных свойств, которые превращают ее в идеальный промежуточный язык при трансляции. Во-первых, вычисление выражения, записанного в обратной польской записи, может проводиться путем однократного просмотра, что является весьма удобным при генерации объектного кода программ. Например, вычисление выражения (2) может быть проведено следующим образом:

# п/п	Анализируемая строка	Действие
0	A B + C D + * E -	r1=A+B
1	r1 C D + * E -	r2=C+D
2	r1 r2 * E -	r1=r1*r2
3	r1 E -	r1=r1-E
4	r1	Вычисление окончено

Здесь r1, r2 - вспомогательные переменные.

Во-вторых, получение обратной польской записи из исходного выражения может осуществляться весьма просто на основе простого алгоритма, предложенного Дейкстры. Для этого вводится понятие стекового приоритета операций (табл. 1):

Операция	Приоритет
(	1
)	1
+ -	2
* /	3
^	4

Таблица 1

Просматривается исходная строка символов слева направо, операнды переписываются в выходную строку, а знаки операций заносятся в стек на основе следующих соображений:

- если стек пуст, то операция из входной строки переписывается в стек;
- операция выталкивает из стека все операции с большим или равным приоритетом в выходную строку;
- если очередной символ из исходной строки есть открывающая скобка, то он проталкивается в стек;
- закрывающая круглая скобка выталкивает все операции из стека до ближайшей открывающей скобки, сами скобки в выходную строку не переписываются, а уничтожают друг друга.

Процесс получения обратной польской записи выражения (1) схематично представлен на Рис. 2:

Просматриваемый символ	1	2	3	4	5	6	7	8	9	10	11	12	13	
Входная строка	(	A	+	B	)	*	(	C	+	D	)	-	E	
Состояние стека	(	(	+	+		*	(	(	+	+	*	-	-	
Выходная строка		A		B	+			C		D	+	*	E	-

Рис. 2

Задание на лабораторную работу

Написать программу-аналог калькулятора (с некоторыми вариациями, в зависимости от желаемой оценки).

Входные данные: произвольная строка символов.

Выходные данные: заключение о корректности входной строки как арифметического выражения, вычисленное значение этого выражения, в случае некорректной входной строки информацию о типе ошибки (например, «Несоответствие количества (и)»). Выходные данные зависят от сложности задания.

Синтаксический анализатор должен распознавать следующие лексеммы:

1. Знаки арифметических операций: '+', '-', '*', '/'
2. Скобки '(' и ')'
3. Действительные числа (т.е. дробные). Например: 12, 66.6, .54, 221.
4. Имя встроенной функции (см. задание максимум)

Задание минимум (на 3): считать строку символов, дать заключение о её соответствии арифметическому выражению, в случае несоответствия указать причину.

Задание максимум (на 5): вычислить значение введённого выражения с учётом приоритета операций. При этом необходимо обрабатывать исключительные ситуации (например, деление на 0). Кроме того, нужно реализовать поддержку одной встроенной функции от двух переменных (например, $\log(a, b)$). При этом аргументы функции сами являются выражениями (в том числе содержат эту функцию). Должна быть реализована возможность сравнительно простого изменения встроенной функции по требованию преподавателя (например, заменить $\log(a, b)$ на $\text{row}(a, b)$). Встроенная функция имеет наивысший приоритет. Скобки служат для изменения порядка действий (обычная математика).

Вне зависимости от сложности, программная реализация должна соответствовать требованиям, изложенным в файле **Преамбула.doc**

Синтаксический анализ и вычисление значения выражения могут быть реализованы с помощью любого алгоритма (например, с помощью метода рекурсивного спуска и т.п.)

Литература

При подготовке данной лабораторной работы были использованы материалы сайта www.algolist.manual.ru

Лабораторная работа №2

Конечные детерминированные автоматы. Преобразование недетерминированного конечного автомата к детерминированному

Теоретическая часть

Конечный автомат представляет собой пятерку

$$A = (Q, T, t, q_0, F)$$

где, Q - конечное множество состояний автомата,

T - непустое конечное множество входных символов, входной алфавит,

$q_0 \in Q$ - начальное состояние,

$F \subseteq Q$ - непустое множество заключительных состояний автомата,

t - переходная функция

$$t: Q \times T \rightarrow R(Q)$$

Такой автомат работает как распознаватель следующим образом (рис.1). На ленте записана входная строка, ограниченная с двух сторон концевыми маркерами. Управляющее устройство находится в начальном состоянии q_0 . Входная лента последовательно просматривается слева направо по одному символу. Допустим, что с помощью считывающей головки оказались просмотрены символы $a_0 a_1 \dots a_i$ и управляющее устройство находится в состоянии q_i . Символ a_i допускается, если существует хотя бы одно состояние $t(q, a_i) \in Q$.

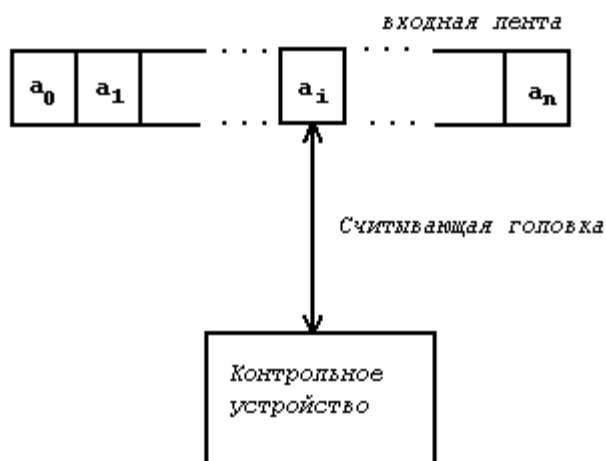


Рис. 1

Если такого состояния не существует, автомат останавливает свою работу. В общем случае таких состояний может быть несколько, и распознаватель будет недетерминированным, т.е. речь в этом случае будет идти о *недетерминированном конечном автомате*.

Строка $x = a_1 a_2 \dots a_n \in T^+$ допускается автоматом, если существует последовательность состояний $q_0, q_1, \dots, q_n$, такая, что $q_{i+1} \in t(q_i, a_{i+1})$, где $i = 0, 1, \dots, n-1$, $q_n \in F$ и после прочтения последнего символа a_n строки был встречен

концевой маркер. Предполагается, что начальный маркер читается в состоянии q_0 . Этот факт можно записать с помощью функции перехода следующим образом: $t(q_0, x) \sqsubseteq F$. В этом случае функцию перехода, соответствующую одному такту работы распознавателя, можно рассматривать, как частный случай ее общей формы записи для строки x , содержащей один символ $x \sqsubseteq a_i$ и состояния q_i . Таким образом, функцию t можно определить рекурсивно $t(q, x) \sqsubseteq t(t(q, a), y)$, если строка $x \sqsubseteq ay$, где $a \sqsubseteq T$ и $y \sqsubseteq T^*$.

Множество всех строк $T(A)$, допускаемых автоматом A , может быть записано в виде

$$T(A) \sqsubseteq \{x \mid x \sqsubseteq T^*, t(q_0, x) \sqsubseteq F\}$$

Практика показала, что одним из удобных способов задания функции переходов является *граф переходов* или его еще называют *граф состояний*. Граф состояний (V, E) является направленным графом и строится следующим образом:

Каждому состоянию множества Q соответствует одна вершина множества V .

Все вершины, кроме заключительных, обозначаются на рисунках в виде окружностей.

На начальную вершину, соответствующую начальному состоянию автомата, указывает стрелка.

Заключительная вершина изображается прямоугольником.

Ребро $(B, C) \sqsubseteq E$ существует на графе, если для символа $a \sqsubseteq T$ и состояния $B \sqsubseteq Q$ значение функции $t(B, a)$ не пусто и равно $C \sqsubseteq Q$.

На рис. 2 изображен граф состояний автомата A .

$A \sqsubseteq (\{q_0, q_1, q_2, q_3, q_4\}, \{a, b, c\}, t, q, \{q_3, q_4\})$, где

$$t(q_0, a) \sqsubseteq q_1$$

$$t(q_0, b) \sqsubseteq q_2$$

$$t(q_1, b) \sqsubseteq q_4$$

$$t(q_2, c) \sqsubseteq q_3$$

$$t(q_2, a) \sqsubseteq q_4$$

$$t(q_4, c) \sqsubseteq q_3$$

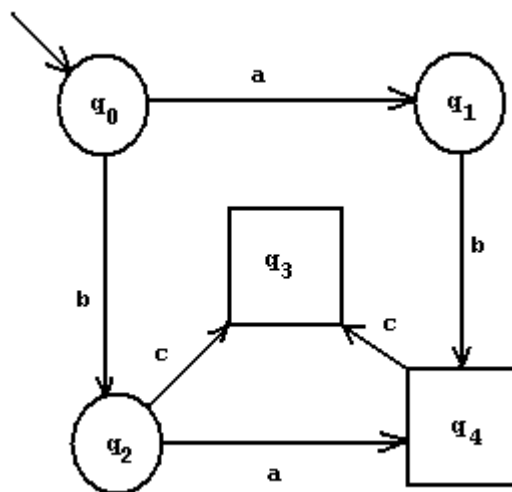


Рис. 2

Рассмотрим еще один пример грамматики, порождающей следующие предложения языка

$$a, a0, a00, \dots, b0, b00, b00, \dots$$

Построим для этой грамматики конечный автомат. Граф его состояний приведен на рис. 3.

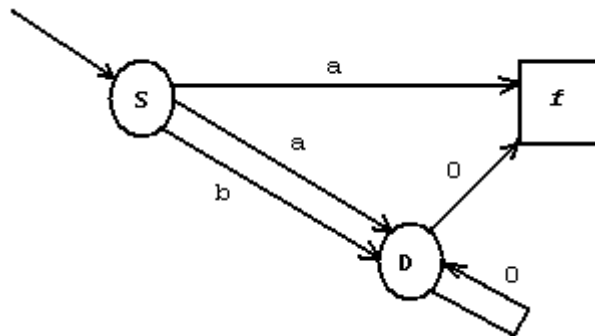


Рис. 3

По графу видно, что конечный автомат недетерминированный, поскольку из состояния S выходят два ребра с символом a . Функция перехода для символа a в состоянии S имеет два значения.

$$t(S, a) \subseteq \{f, D\}$$

Как известно, недетерминированный автомат достаточно сложно использовать для разбора. Поэтому интересует возможность преобразования недетерминированного автомата в детерминированный. Для конечных автоматов этот вопрос решается положительно. Доказано, что, если A - недетерминированный автомат, то существует детерминированный автомат A' , такой, что $T(A) = T(A')$. Допустим, что функция переходов $t(q, a)$ имеет множество значений $Q \subseteq Q$. Будем считать это множество одним новым состоянием q' . Присоединим это состояние к множеству состояний Q автомата, т.е. $Q \subseteq Q \cup q'$. Определим функцию перехода для нового состояния q' .

$$t(q \cup a) \subseteq \bigcup_{i \in Q} t(i, a) \text{ для всех } a \in T$$

Выполнив данное преобразование для всех функций перехода, имеющих не единственное значение, получим детерминированный автомат. Запишем для приведенного выше примера все функции перехода.

$$\begin{aligned} t(S, a) &\subseteq \{f, D\} \\ t(S, b) &\subseteq D \\ t(D, 0) &\subseteq D \\ t(D, 0) &\subseteq \{f\} \end{aligned}$$

Введем новое состояние $q \subseteq \{D, f\}$ и перепишем функции перехода.

$$\begin{aligned} t(S, a) &\subseteq q \\ t(S, b) &\subseteq D \\ t(D, 0) &\subseteq q \end{aligned}$$

$t(q,0) \neq q$ так как одно состояние из $\{D, f\}$ f - конечное, а $t(D,0) \neq q$

Граф состояний полученного детерминированного автомата показан на рис. 4.

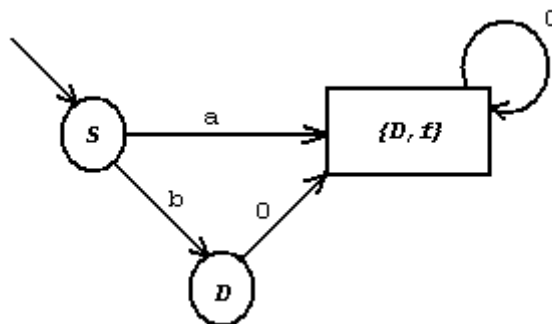


Рис. 4

Следует заметить, что детерминирование автомата не должно изменить множество предложений (или входных строк), которые этот автомат разбирает.

Задание на лабораторную работу

Написать программу, реализующую работу конечного автомата.

Входные данные:

1. текстовый файл, описывающий граф переходов конечного автомата. Файл представляет собой набор строк. В каждой строке задаётся **одно** правило перехода в следующем виде:

$$q \langle N \rangle, \langle C \rangle = \langle q | f \rangle \langle N \rangle$$

Здесь символ q обозначает состояние автомата, f – конечное состояние автомата, $\langle N \rangle$ - произвольное число, обозначающее номер состояния, $\langle C \rangle$ - **один** символ.

Пример: $q12, g = f0$ – запись означает, что если автомат находится в состоянии №12 и читает с ленты символ 'g', то он перейдёт в конечное состояние с номером 0

Дополнительные условия

Количество строк в файле (возможных переходов) – неограниченное количество

Начальное состояние автомата (с которого начинается его работа) – $q0$

Строки в файле не обязаны быть отсортированы по какому-либо критерию

Состояния автомата необязательно нумеруются последовательно

2. строка символов, которую нужно проанализировать с помощью построенного автомата и дать заключение о возможности (или невозможности) разбора этой строки с помощью данного автомата

Пример: строки ab , abc , ba допускаются автоматом, граф переходов которого изображён на рис. 2. Строки b , ak , bad этим автоматом не допускаются.

Выходные данные:

1. Заключение о детерминированности или недетерминированности заданного автомата.
2. В случае недетерминированного автомата вывести переходы для соответствующего ему детерминированного автомата (в виде, соответствующем входному файлу).
3. Заключение о возможности (невозможности) разбора автоматом введённой строки символов

Задание минимум (на 3): считать из файла автомат (файл не содержит синтаксических ошибок, заданный с его помощью автомат детерминирован), дать заключение о возможности (невозможности) разбора этим автоматом введённой строки.

Задание максимум (на 5. В принципе, совершенству нет предела, но тем не менее): считать из файла автомат (файл может содержать синтаксические ошибки, заданный с его помощью автомат недетерминирован, возможно, граф переходов содержит висячие вершины), вывести информацию об автомате (детерминирован/нет, всё, что угодно), детерминировать автомат, вывести таблицу переходов для нового автомата, разобрать входную строку и дать заключение о возможности/невозможности её разбора.

Весьма неплохо было бы при написании программы использовать возможности **объектно-ориентированного** программирования. В качестве дополнения, например, можно графически представить граф переходов для автомата до и после проведения операции детерминирования.

При возникновении неоднозначности в оценке, может быть предложено в качестве дополнительного задания сделать файл для автомата, разбирающего какую-то строку (например, `if (a>b) exit(1);`).

Литература

При подготовке данной лабораторной работы были использованы лекции Тимофеева П.А. по курсу «Теория алгоритмических языков и компиляторов».

Приложения

1. Пример входного файла для автомата, разбирающего строку `for ([abc] = [0-9]; [abc] [<|>] [0-9]; [abc]++)`

Здесь `[abc]` – строка произвольной длины, состоящая из символов `a, b, c, A, B, C`

`[0-9]` – строка произвольной длины, состоящая из символов цифр

`[<|>]` – или символ “<”, или символ “>”

q0, =q0
q0,f=q1
q1,o=q2
q2,r=q3
q3, =q3
q3,(=q4
q4, =q4
q4,i=q5
q5,n=q6
q6,t=q7
q7, =q7
q7,a=q8
q7,b=q8
q7,c=q8
q7,A=q8
q7,B=q8
q7,C=q8
q8,a=q8
q8,b=q8
q8,c=q8
q8,A=q8
q8,B=q8
q8,C=q8
q8,==q10
q8, =q9
q9, =q9
q9,==q10
q10, =q10
q10,0=q11
q10,1=q11
q10,2=q11
q10,3=q11
q10,4=q11
q10,5=q11
q10,6=q11
q10,7=q11
q10,8=q11
q10,9=q11
q11,0=q11
q11,1=q11
q11,2=q11
q11,3=q11
q11,4=q11
q11,5=q11
q11,6=q11
q11,7=q11
q11,8=q11
q11,9=q11

q11,;=q13
q11, =q12
q12, =q12
q12,;=q13
q13, =q13
q13,a=q14
q13,b=q14
q13,c=q14
q13,A=q14
q13,B=q14
q13,C=q14
q14,a=q14
q14,b=q14
q14,c=q14
q14,A=q14
q14,B=q14
q14,C=q14
q14, =q15
q14,<=q16
q14,>=q16
q15, =q15
q15,<=q16
q15,>=q16
q16, =q16
q16,0=q17
q16,1=q17
q16,2=q17
q16,3=q17
q16,4=q17
q16,5=q17
q16,6=q17
q16,7=q17
q16,8=q17
q16,9=q17
q17,0=q17
q17,1=q17
q17,2=q17
q17,3=q17
q17,4=q17
q17,5=q17
q17,6=q17
q17,7=q17
q17,8=q17
q17,9=q17
q17, =q18
q17,;=q19
q18, =q18
q18,;=q19

q19, =q19
q19,a=q20
q19,b=q20
q19,c=q20
q19,A=q20
q19,B=q20
q19,C=q20
q20,a=q20
q20,b=q20
q20,c=q20
q20,A=q20
q20,B=q20
q20,C=q20
q20, =q21
q20,+=q22
q21, =q21
q21,+=q22
q22,+=q23
q23, =q23
q23,)=f0

2. Пример работающей программы (операция детерминирования не производится)

```
#include <string>
#include <vector>
#include <algorithm>
#include <iostream>
#include <fstream>

typedef unsigned char uchar;

using namespace std;

class format_error: public runtime_error {
public:
    format_error(const char* msg): runtime_error(msg){}
};

class StateReader{ //class for reading states from file
public:
    struct ElementarySwitch{ // one switch for automat
        int initialState;    // number of current state
        uchar letter;        // reading symbol
        bool isTerminalState; // is next state terminal?
        int nextState;       // number of next state
        ElementarySwitch():
            initialState(-1),
            letter(0),
            isTerminalState(false),
            nextState(-1)
        {}

        // next 2 operators - for sorting states array
        friend bool operator > (const ElementarySwitch& el, const ElementarySwitch& er){
            if (el.initialState > er.initialState) return true;
            if (el.initialState == er.initialState){
                if (el.letter > er.letter) return true;
                if (el.letter == er.letter){
                    if (el.isTerminalState != er.isTerminalState) return er.isTerminalState;
                    if (el.nextState > er.nextState) return true;
                }
            }
            return false;
        }

        friend bool operator < (const ElementarySwitch& el, const ElementarySwitch& er){
            return !operator >(el, er);
        }
    };

    typedef vector<ElementarySwitch> StatesSwitchArray;

    StateReader(const char* filename);
    ~StateReader() {stateFile.close();}

protected:
    StatesSwitchArray statemachineStates; // array of switches for automat
```

```

private:
    ifstream stateFile; // stream for reading file
};

StateReader::StateReader(const char* filename):stateFile(filename){
    if(!stateFile.is_open()) throw runtime_error("Invalid states file"); // can't open file

    string tmpStr;
    while(getline(stateFile, tmpStr)){
        if (tmpStr.size() == 0) continue; // skip empty string
        // several check for input file format
        if (tmpStr[0] != 'q' && tmpStr[0] != 'Q')
            throw format_error("Line must begin with 'q' letter");
        string::size_type commaPos = tmpStr.find(',');
        if (commaPos == string::npos)
            throw format_error("There is no comma");
        string stateNumber = tmpStr.substr(1, commaPos - 1);
        ElementarySwitch tmpSw; // prepare next element in array
        tmpSw.initialState = atoi(stateNumber.c_str());
        if (tmpSw.initialState == 0 && stateNumber[0] != '0')
            throw format_error("State number must contains digits only");
        tmpSw.letter = tmpStr[commaPos + 1];
        if (tmpStr[commaPos + 2] != '=')
            throw format_error("Expected '=' sign");
        switch (tmpStr[commaPos + 3]){
            case 'f':
            case 'F':
                tmpSw.isTerminalState = true;
                break;
            case 'q':
            case 'Q':
                tmpSw.isTerminalState = false;
                break;
            default:
                throw format_error("Next state must begin with 'q' or 'f' letter");
        }
        stateNumber = tmpStr.substr(commaPos + 4);
        tmpSw.nextState = atoi(stateNumber.c_str());
        if (tmpSw.nextState == 0 && stateNumber[0] != '0')
            throw format_error("State number must contains digits only");

        statemachineStates.push_back(tmpSw); // add one switch to array of switches
    }
}

class StateMachine: public StateReader{
public:
    StateMachine(const char* filename);
    bool isDeterministic() const {return deterministic;}
    bool hasHangs() const {return hangs;} // means that graph contains isolated node(s)
    const StateReader::StatesSwitchArray& GetSwitches() const {return statemachineStates;} // just for
    printing
    bool isExpressionCorrect(const string& expression, int& errorPos);

protected:
    void SortStates();
    bool _isDeterministic();

```

```

bool _hasHangs();
int _findNextIndex(int curState, uchar sym);

private:
    bool deterministic;
    bool hangs;
};

StateMachine::StateMachine(const char* filename):
    StateReader(filename),
    deterministic(true),
    hangs(false)
{
    SortStates();
    //some little check of machine
    if (statemachineStates.size() == 0)
        throw runtime_error("Automat is empty");
    if (statemachineStates[0].initialState != 0)
        throw runtime_error("There is no initial state");
    size_t ln = statemachineStates.size();
    bool hasFinalState = false;
    for (size_t i = 0; i < ln; i++)
        if (hasFinalState = statemachineStates[i].isTerminalState)
            break;
    if (!hasFinalState)
        throw runtime_error("There is no final state");

    deterministic = _isDeterministic(); // check if automat is deterministic
    hangs = _hasHangs(); // check if may be hangs
}

bool StateMachine::_isDeterministic(){
    size_t ln = statemachineStates.size(); // count of elements in array
    bool isDet = true;
    for (size_t i = 1; i < ln; i++)
        if (statemachineStates[i-1].initialState == statemachineStates[i].initialState &&
            statemachineStates[i-1].letter == statemachineStates[i].letter &&
            (statemachineStates[i-1].isTerminalState != statemachineStates[i].isTerminalState ||
             statemachineStates[i-1].nextState != statemachineStates[i].nextState))
        {
            isDet = false;
            break;
        };
};

return isDet;
}

bool StateMachine::_hasHangs(){
    size_t ln = statemachineStates.size();
    bool isHangs = false;
    for (size_t i = 0; i < ln; i++){
        if (!statemachineStates[i].isTerminalState){
            bool found = false;
            // very bad algorithm to search in _SORTED_ array. I was laziness to do better :->
            for (size_t j = 0; j < ln; j++){
                if (statemachineStates[i].nextState == statemachineStates[j].initialState){
                    found = true;
                }
            }
        }
    }
}

```

```

        break;
    }
}
if (!found){
    isHangs = true;
    break;
}
}
}
return isHangs;
}

void StateMachine::SortStates(){
    sort(statemachineStates.begin(), statemachineStates.end()); // common sorting algorithm from
<algorithm>
}

int StateMachine::_findNextIndex(int curState, uchar sym){
    int found = -1;
    size_t ln = statemachineStates.size();

    // very bad algorithm to search in _SORTED_ array
    for (size_t j = 0; j < ln; j++){
        if (statemachineStates[j].initialState == curState &&
            statemachineStates[j].letter == sym){
            found = j;
            break;
        }
    }
    return found;
}

bool StateMachine::isExpressionCorrect(const string& expression, int& errorPos){
    if ( !deterministic || hangs)
        throw runtime_error("This automat cannot check expression");
    // emulate automat's task
    int currentState = 0;
    size_t strLen = expression.size();
    for (int i = 0; i < strLen; i++){
        int idx = _findNextIndex(currentState, expression[i]);
        if (idx < 0){
            errorPos = i;
            return false;
        }
        if (statemachineStates[idx].isTerminalState){
            if (i == strLen - 1) return true;
            errorPos = i + 1;
            return false;
        }
        currentState = statemachineStates[idx].nextState;
    }
    errorPos = strLen;
    return false;
}

int _tmain(int argc, _TCHAR* argv[]) {
    try{ // try to create object of StateMachine

```

```

    StateMachine sr("states.txt");
    StateReader::StatesSwitchArray::const_iterator it; // another way to access to array's elements
    for (it = sr.GetSwitches().begin(); it != sr.GetSwitches().end(); it++)
        cout << "q" << it->initialState << ", " << it->letter << "=" << (it->isTerminalState ? "f" : "q") << it-
>nextState << endl;
    cout << "There are" << (sr.hasHangs() ? "" : "n't") << " hangs" << endl;
    cout << "Automat is" << (sr.isDeterministic() ? "" : "n't") << " deterministic" << endl;

    string testExpr;
    cout << "Please, enter expression to check ";
    cin >> testExpr;
    // for test with related "states.txt" file
    // testExpr = " for( int abAccc= 943 ; a<478; bbc++ )";
    // cout << testExpr << endl;
    int err;
    bool res = sr.isExpressionCorrect(testExpr, err);
    if (res) cout << "Expression is correct!" << endl;
    else cout << "Incorrect expression. Error position: " << err << endl;
}
catch(const exception& err){
    cerr << err.what() << endl;
}
return 0;
}

// Mark for such realization will be 4

```

Лабораторная работа №3

Недетерминированные магазинные автоматы.

Теоретическая часть

Модель магазинного автомата (рис.1) состоит из

- ♦ входной ленты,
- ♦ устройства управления и
- ♦ вспомогательной ленты, называемой магазином или стеком.

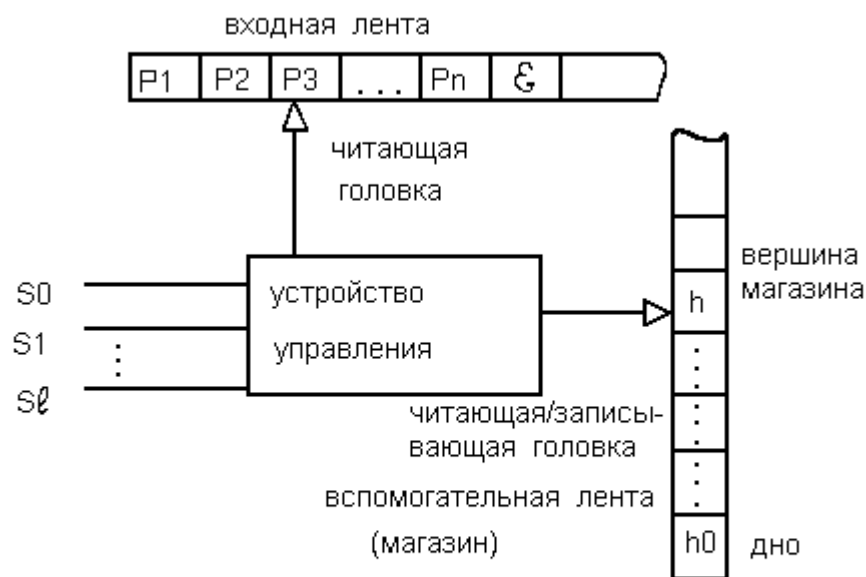


Рис.1. Модель магазинного автомата

Входная лента разделяется на клетки (позиции), в каждой из которых может быть записан символ входного алфавита. При этом предполагается, что в неиспользуемых клетках входной ленты расположены пустые символы ϵ .

Вспомогательная лента также разделена на клетки, в которых могут располагаться символы магазинного алфавита. Начало вспомогательной ленты называется дном магазина. Связь устройства управления с лентами осуществляется двумя головками, которые могут перемещаться вдоль лент.

Головка входной ленты может перемещаться только в одну сторону - вправо или оставаться на месте. Она может выполнять только чтение. Головка вспомогательной ленты способна выполнять как чтение, так и запись, но эти операции связаны с перемещением головки определенным образом:

- ♦ при записи головка предварительно сдвигается на одну позицию вверх, а затем символ заносится на ленту,
- ♦ при чтении символ, находящийся под головкой считывается с ленты, а затем головка сдвигается на одну позицию вниз, т.о. головка всегда установлена против последнего записанного символа. Позицию, находящуюся в рассматриваемый момент времени под головкой, называют вершиной магазина.

Определение. Магазинный автомат M определяется следующей совокупностью семи объектов: $M = \{S, P, Z, \hookrightarrow, s_0, h_0, F\}$, где

S - алфавит состояний,

P - входной алфавит,

Z - алфавит **магазина символов**, записываемых на вспомогательную ленту,

$\hookrightarrow$ - функция переходов, $\hookrightarrow : \{S \times P \downarrow \{\epsilon\} \times H\} \downarrow \square S \times H^*$, если M -автомат - детерминированный, и $\hookrightarrow : \{S \times P \downarrow \{\epsilon\} \times H\} \downarrow \square 2^{S \times H^*}$, если M -автомат - недетерминированный.

s_0 - начальное состояние, $s_0 \uparrow S$.

h_0 - маркер дна, он всегда записывается на дно магазина, $h_0 \uparrow H$.

F - множество конечных состояний. F является подмножеством S .

Функция $\hookrightarrow$ отображает тройки (p_i, s_j, h_k) в пары $(s_r, \blacktriangleright)$, где $\blacktriangleright \uparrow H^*$ и h_k - символ в вершине магазина, для детерминированного автомата или в множество таких пар для недетерминированного автомата.

Эта функция описывает изменение состояния магазинного автомата, происходящее при чтении символа с входной ленты и перемещении входной головки.

В дальнейшем при построении магазинных автоматов потребуются две разновидности функций переходов:

1) функция переходов с пустым символом в качестве входного символа:

$\hookrightarrow^0(s, \epsilon, h) = (s', \hookrightarrow)$, которая, независимо от того, какой символ находится под читающей головкой входной ленты, предписывает прочитать символ h из вершины магазина, изменить состояние автомата на s' и записать цепочку $\hookrightarrow$ в магазин.

2) функция переходов с определенным входным символом:

$\hookrightarrow(s, a, \square \square h) = (s', \hookrightarrow)$, которая предписывает прочитать входной символ a , изменить состояние автомата на s' и заменить верхний символ магазина h цепочкой $\hookrightarrow$.

Работа магазинного автомата

Чтобы описать, как работает автомат, введем понятие конфигурации.

Определение. Конфигурацией автомата M называют тройку $(s, \Rightarrow, \rightarrow) \uparrow S \times P^* \times H^*$, где

s - текущее состояние управляющего устройства,

$\Rightarrow$ - неиспользованная часть входной цепочки $\Rightarrow \uparrow P^*$, самый левый символ этой цепочки находится под головкой. Если $a = \epsilon$, то считается, что вся входная цепочка прочитана.

$\rightarrow$ - цепочка, записанная в магазине, $\rightarrow \uparrow H^*$, самый правый символ цепочки считается вершиной магазина. Если $\rightarrow = \epsilon$, то магазин пуст.

Работа автомата может быть представлена как смена конфигураций. Один такт работы автомата заключается в определении новой конфигурации по заданной. Это записывается так:

$$(s, a\Rightarrow, \rightarrow h) \vdash (s', \Rightarrow, \rightarrow \hookrightarrow)$$

Такая смена конфигураций возможна, если функция $\hookrightarrow(s, a, h)$ (или $\hookrightarrow(s, \epsilon, h)$) определена и имеет значение $(s', \hookrightarrow)$. При этом предполагается, что автомат

- читает символ a , находящийся под головкой. Или не читает ничего, в случае входного символа ϵ .
- определяет новое состояние s'
- читает символ h , находящийся в вершине магазина и
- записывает цепочку $\Leftarrow$ в магазин вместо символа h . Если $\Leftarrow = \epsilon$, то верхний символ оказывается удаленным из магазина.

Таким образом, могут быть три случая при работе автомата:

- $\Leftarrow(s, a, h)$ определена и выполняется такт работы,
- $\Leftarrow(s, a, h)$ не определена, но определена функция $\Leftarrow(s, \epsilon, h)$ и выполняется пустой такт (без чтения входной информации).
- функции $\Leftarrow(s, a, h)$ и $\Leftarrow(s, \epsilon, h)$ не определены, в этом случае дальнейшая работа автомата невозможна.

Начальной конфигурацией называется конфигурация $(s_0, \Rightarrow, h_0)$, где s_0 - начальное состояние, $\Rightarrow$ - исходная цепочка, h_0 - маркер дна магазина.

Заключительная конфигурация – $(s, \Leftarrow, \Leftarrow)$, где s принадлежит множеству заключительных состояний F .

Для обозначения последовательности сменяющих друг друга конфигураций условимся использовать знак $\vdash^*$. Таким образом последовательность

$$(s_1, \Rightarrow_1, \rightarrow_1) \vdash (s_2, \Rightarrow_2, \rightarrow_2) \vdash \dots \vdash (s_n, \Rightarrow_n, \rightarrow_n)$$

записывается в сокращенном виде как:

$$(s_1, \Rightarrow_1, \rightarrow_1) \vdash^* (s_n, \Rightarrow_n, \rightarrow_n).$$

Язык, допускаемый магазинным автоматом

Определение. Цепочка $\Rightarrow$ называется **допустимой** для автомата M , если существует последовательность конфигураций, в которой первая конфигурация является начальной с цепочкой $\Rightarrow$, а последняя – заключительной. $(s_0, \Rightarrow, h_0) \vdash^* (s_1, \Leftarrow, \Leftarrow)$, где $s_1 \uparrow F$.

Определение. Множество цепочек, допускаемых автоматом M , называется языком, **допускаемым** или определяемым автоматом M , и обозначается $L(M)$.

$$L(M) = \{ \Rightarrow \mid (s_0, \Rightarrow, h_0) \vdash^* (s', \Leftarrow, \Leftarrow) \text{ и } s' \uparrow F \}$$

Чтобы лучше представить себе работу магазинного автомата, рассмотрим два примера. Пусть задан магазинный автомат M_1 в следующем виде:

$$\begin{aligned} M_1: P &= \{a, b\}; S = \{s_0, s_1, s_2\}; Z = \{h_0, a\}; F = \{s_0\}; \\ \Leftarrow(s_0, a, h_0) &= (s_1, h_0a), \\ \Leftarrow(s_1, a, a) &= (s_1, aa), \\ \Leftarrow(s_1, b, a) &= (s_2, \Leftarrow), \\ \Leftarrow(s_2, b, a) &= (s_2, \Leftarrow), \\ \Leftarrow(s_2, \Leftarrow, h_0) &= (s_0, \Leftarrow). \end{aligned}$$

Этот автомат является **детерминированным**, поскольку каждому набору аргументов соответствует единственное значение функции. Работу автомата при

распознавании входной цепочки aabb можно представить в виде последовательности конфигураций:

$$(s_0, aabb, h_0) \vdash (s_1, abb, h_0a) \vdash (s_1, bb, h_0aa) \vdash (s_2, b, h_0a) \vdash (s_2, \epsilon, h_0) \vdash (s_0, \epsilon, \epsilon).$$

Нетрудно проверить, что при задании входной цепочки aabb автомат не сможет закончить работу. Следовательно, эта цепочка не принадлежит языку, допускаемому автоматом M_1 .

Магазинный автомат M_2 , заданный следующим описанием:

$$M_2: P = \{a, b\}; S = \{s_0, s_1, s_2\}; Z = \{h_0, a, b\}; F = \{s_2\};$$

$$(1) \Leftrightarrow (s_0, a, h_0) = (s_0, h_0a),$$

$$(2) \Leftrightarrow (s_0, b, h_0) = (s_0, h_0b),$$

$$(3) \Leftrightarrow (s_0, a, a) = \{(s_0, aa), (s_1, \epsilon)\},$$

$$(4) \Leftrightarrow (s_0, b, a) = (s_0, ab),$$

$$(5) \Leftrightarrow (s_0, a, b) = (s_0, ba),$$

$$(6) \Leftrightarrow (s_0, b, b) = \{(s_0, bb), (s_1, \epsilon)\},$$

$$(7) \Leftrightarrow (s_1, a, a) = (s_1, \epsilon),$$

$$(8) \Leftrightarrow (s_1, b, b) = (s_1, \epsilon),$$

$$(9) \Leftrightarrow (s_2, \epsilon, h_0) = (s_2, \epsilon),$$

является **недетерминированным** автоматом, поскольку одному и тому же набору аргументов, например (s_0, a, a) , соответствуют два значения функции. Работу автомата рассмотрим для входной цепочки abba. Если использовать последовательность команд (1),(4),(6.1),(5), то получим последовательность конфигураций:

$$\begin{array}{l} (s_0, abba, h_0) \\ \vdash (s_0, bba, h_0a), \quad (1) \\ \vdash (s_0, ba, h_0ab), \quad (4) \\ \vdash (s_0, a, h_0abb), \quad (6.1) \\ \vdash (s_0, \epsilon, h_0abba). \quad (5) \end{array}$$

которая показывает, что дальнейшая работа невозможна, т.к. входная цепочка прочитана и переход (s_0, ϵ, h_0abba) не определен. Если же использовать последовательность команд (1),(4),(6.2),(3),(9), то получим заключительную конфигурацию:

$$\begin{array}{l} (s_0, abba, h_0) \\ \vdash (s_0, bba, h_0a), \quad (1) \\ \vdash (s_0, ba, h_0ab), \quad (4) \\ \vdash (s_1, a, h_0a), \quad (6.2) \\ \vdash (s_1, \epsilon, h_0), \quad (3) \\ \vdash (s_2, \epsilon, \epsilon). \quad (9). \end{array}$$

Т.о. можно сделать вывод о том, что цепочка abba допускается автоматом M_2 .

Построение магазинного автомата

Пусть задана грамматика $G = \{VN, VT, I, P\}$. Определим компоненты автомата M следующим образом:

$$S = \{s_0\}, P = VT, Z = VN \Downarrow VT \Downarrow \{h_0\}, F = \{s_0\},$$

в качестве начального состояния автомата примем s_0 и построим функцию переходов так:

1. Для всех $A \rightarrow VN$ таких, что встречаются в левой части правил $A \rightarrow \dots$, построим команды вида:
 $\Rightarrow^0(s_0, \leftarrow, A) = (s_0, \Rightarrow^R)$,
 где $\Rightarrow^R$ – реверс цепочки $\Rightarrow$.
2. Для всех $a \rightarrow VT$ построим команды вида
 $\Rightarrow(s_0, a, a) = (s_0, \leftarrow)$
3. Для перехода в конечное состояние построим команду
 $\Rightarrow(s_0, \leftarrow, h_0) = (s_0, \leftarrow)$
4. Начальную конфигурацию автомата определим в виде:
 $(s_0, \triangleright, h_0 I)$,
 где $\triangleright$ – $\square$ исходная цепочка, записанная на входной ленте.

Автомат, построенный по приведенным выше правилам, работает следующим образом. Если в вершине магазина находится терминал, и символ, читаемый с входной ленты, совпадает с ним, то по команде типа (2) терминал удаляется из магазина, а входная головка сдвигается. Если же в вершине магазина находится нетерминал, то выполняется команда типа (1), которая вместо терминала записывает в магазин цепочку, представляющую собой правую часть правила грамматики. Следовательно, автомат, последовательно заменяя нетерминалы, появляющиеся в вершине магазина, строит в магазине левый вывод входной цепочки, удаляя полученные терминальные символы, совпадающие с символами входной цепочки. Это означает, что каждая цепочка, которая может быть получена с помощью левого вывода в грамматике G , допускается построенным автоматом M .

Пример построения автомата

Процедуру построения автомата рассмотрим на примере грамматики с правилами:

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid a \end{aligned}$$

Для искомого автомата имеем:

$$P = \{a, +, *, (,)\}, Z = \{E, T, F, a, +, *\}, h_0, (\}, S = \{s_0\}, F = \{s_0\}$$

Для всех правил грамматики строим команды типа (1):

$$(1) \Rightarrow^0(s_0, \Rightarrow, E) = \{(s_0, T+E); (s_0, T)\},$$

$$(2) \Rightarrow^0(s_0, \leftarrow, T) = \{(s_0, F*T); (s_0, F)\},$$

$$(3) \Rightarrow^0(s_0, \leftarrow, F) = \{(s_0, (E)); (s_0, a)\},$$

Для всех терминальных символов строим команды типа (2):

$$(4) \Rightarrow(s_0, a, a) = (s_0, \leftarrow),$$

$$(5) \Rightarrow(s_0, +, +) = (s_0, \leftarrow),$$

$$(6) \Rightarrow(s_0, *, *) = (s_0, \leftarrow),$$

$$(7) \Rightarrow(s_0, (, () = (s_0, \leftarrow),$$

$$(8) \Rightarrow(s_0,), ()) = (s_0, \leftarrow),$$

Для перехода в конечное состояние построим команду:

$$(9) \Rightarrow(s_0, \leftarrow, h_0) = (s_0, \leftarrow).$$

Построенный автомат является недетерминированным.

Начальную конфигурацию с цепочкой $a + a^*a$ запишем так: $(s_0, a+a^*a, h_0E)$.

Последовательность тактов работы построенного автомата, показывающая, что заданная цепочка допустима, имеет вид:

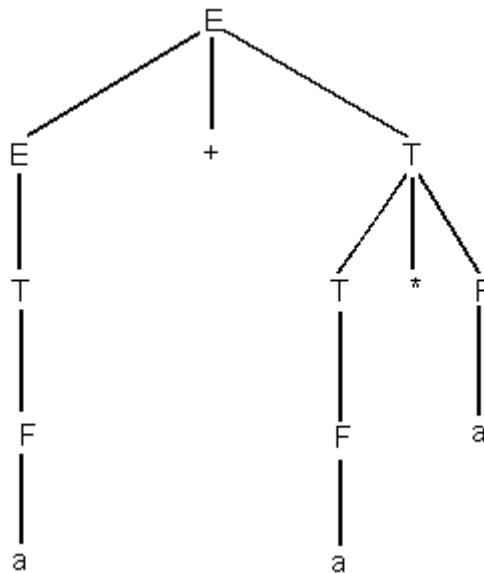
$$\begin{aligned} (s_0, a+a^*a, h_0E) &\vdash (s_0, a+a^*a, h_0T+E) \vdash (s_0, a+a^*a, h_0T+T) \vdash (s_0, a+a^*a, h_0T+F) \vdash \\ (s_0, a+a^*a, h_0T+a) &\vdash (s_0, +a^*a, h_0T+) \vdash (s_0, a^*a, h_0T) \vdash (s_0, a^*a, h_0F^*T) \vdash \\ (s_0, a^*a, h_0F^*F) &\vdash (s_0, a^*a, h_0F^*a) \vdash (s_0, ^*a, h_0F^*) \vdash (s_0, a, h_0F) \vdash \\ (s_0, a, h_0a) &\vdash (s_0, \surd, h_0) \vdash (s_0, \surd, \surd). \end{aligned}$$

Отметим, что последовательность правил, используемая построенным автоматом, соответствует левому выводу входной цепочки:

$E \Rightarrow E+T \Rightarrow T+T \Rightarrow F+T \Rightarrow a+T \Rightarrow a+T^*F \Rightarrow a+F^*F \Rightarrow a+a^*F \Rightarrow \square a+a^*a$.

Если по такому выводу строить дерево, то построение будет происходить сверху вниз, т.е. от корня дерева к листьям.

Такой способ построения дерева по заданной цепочке называется **нисходящим**.



Магазинные автоматы называют часто распознавателями, поскольку они определяют, является ли цепочка, подаваемая на вход автомата, допустимой или нет, и следовательно, отвечают на вопрос, принадлежит ли эта цепочка языку, порождаемому грамматикой, использованной для построения автомата.

Учитывая характер построения вывода в магазине, автоматы рассмотренного типа называют **нисходящими распознавателями**.

Задание на лабораторную работу

Написать программу, реализующую работу недетерминированного магазинного автомата.

Входные данные:

1. Текстовый файл с описанием грамматики, для которой строится магазинный автомат.

Каждая строка в файле может задавать несколько правил грамматики с одинаковой левой частью. В этом случае правила отделяются друг от друга символом '|'.
Для отделения левой части правила от правой используется символ '>'. В одной строке может быть несколько символов '>'. В этом случае первый из них трактуется, как символ, отделяющий правую и левую части продукции, все последующие – как терминальный символ.

Все нетерминалы задаются с помощью прописных букв латинского алфавита. Все символы, не описанные выше, являются терминальными символами. Правил в грамматике (а значит и во входном файле) – неограниченное количество.

Все нетерминалы задаются с помощью прописных букв латинского алфавита.

Все символы, не описанные выше, являются терминальными символами.

Правил в грамматике (а значит и во входном файле) – неограниченное количество.

Рекомендуется считать пробельные символы в файле незначащими, а для терминала «пробел» использовать какой-нибудь другой символ (например, ~, либо заключать пробел в апострофы).

Пример входного файла

$E > \square E + T \mid T$

$T > \square T * F \mid F$

$F > (E) \mid a \mid \sim$

2. Строка символов, которую нужно проанализировать с помощью построенного автомата и дать заключение о допустимости (или недопустимости) автоматом данной цепочки символов.

Выходные данные

1. На оценку «удовлетворительно»: заключение о допустимости (или недопустимости) автоматом цепочки символов.
2. На оценку «отлично»: значение множеств P, Z и т.д.; список команд (1) – (4) (см раздел «Построение магазинного автомата»); цепочка конфигураций магазинного автомата, полученная в процессе его работы; заключение о допустимости (или недопустимости) автоматом цепочки символов.

Литература

При подготовке данной лабораторной работы были использованы материалы сайта <http://mf.grsu.by/>.

Лабораторная работа №4

Разработка транслятора заданных конструкций языка СИ++. Разработка синтаксического анализатора, обнаруживающего максимальное число ошибок.

Теоретическая часть

Любая программа потенциально содержит множество ошибок самого разного уровня. Например, ошибки могут быть:

- ◆ Лексическими, такими как неверно записанные идентификаторы, ключевые слова или операторы;
- ◆ Синтаксическими, например арифметические выражения с несбалансированными скобками;
- ◆ Семантическими, такими как операторы, применяемые к несовместимым с ними операндам;
- ◆ Логическими, например бесконечная рекурсия

Зачастую основные действия по выявлению ошибок и восстановлению после них концентрируются в фазе синтаксического анализа. Одна из причин этого состоит в том, что многие ошибки по природе своей являются синтаксическими или проявляются, когда поток токенов, идущий от лексического анализатора, нарушает определяющие язык программирования грамматические правила.

Обработчик ошибок синтаксического анализатора имеет очень просто формулируемые цели:

- ◆ Он должен ясно и точно сообщать о наличии ошибок;
- ◆ Он должен обеспечивать быстрое восстановление после ошибки, чтобы продолжить поиск следующих ошибок;
- ◆ Он не должен существенно замедлять обработку корректной программы.

Эффективная реализация этих целей представляет собой весьма сложную задачу.

К счастью, обычные ошибки достаточно просты, и для их обработки часто достаточно относительно простых механизмов обработки ошибок. В некоторых случаях, однако, ошибка может произойти задолго до момента её выявления (и за много строк кода до места её выявления), и определить её природу весьма непросто. В некоторых ситуациях обработчик ошибок, по сути, должен попросту догадаться, что именно имел в виду программист, когда писал программу.

Как именно обработчик ошибок должен сообщить о наличии ошибки? Как минимум – сообщить о месте в исходной программе, где была выявлена ошибка, т.к. на самом деле ошибка находится, как правило, в пределах нескольких предыдущих токенов. Общая стратегия, используемая многими компиляторами, заключается в выводе ошибочной строки с указанием позиции, в которой обнаружена ошибка. Если имеется обоснованное предположение о природе ошибки, вывод включает диагностическое пояснение типа «отсутствие точки с запятой в данной позиции».

После того, как ошибка выявлена, в чём же должно состоять восстановление синтаксического анализатора? В большинстве случаев окончание анализа после выявления первой ошибки не является хорошим решением, т.к. восстановление и продолжение анализа могло бы выявить ряд последующих ошибок. Обычно восстановление после ошибки заключается в том, что синтаксический анализатор пытается восстановить некоторое своё состояние, в котором продолжается обработка входного потока так, как если бы он был корректным.

Неадекватное восстановление после ошибки может привести к настоящей лавине ложных ошибок, которые не были допущены программистом, а появились вследствие изменений состояния синтаксического анализатора в процессе восстановления после ошибки. Аналогично восстановление после синтаксической ошибки может привести к ложным семантическим ошибкам, которые выявятся позже, на стадии семантического анализа или генерации кода. Например, при восстановлении после ошибки синтаксический анализатор может пропустить объявление некоторой переменной.

Консервативная стратегия компилятора состоит в запрете сообщений об ошибках, которые возникают во входном потоке слишком близко. После обнаружения одной синтаксической ошибки компилятор должен успешно проанализировать несколько входных токенов и только после этого разрешить вывод других сообщений об ошибках.

Стратегии восстановления после ошибок

Существуют различные стратегии, которые синтаксический анализатор может использовать для восстановления после синтаксической ошибки. Вот некоторые из них:

Режим паники. Эта стратегия реализуется проще всего и может использоваться большинством методов синтаксического анализа. При обнаружении ошибки синтаксический анализатор пропускает входные символы по одному, пока не будет найден один из специально определённого множества синхронизирующих токенов. Обычно такими токенами являются разделители, например `end` или точка с запятой, роль которых в исходной программе совершенно очевидна. Разработчик компилятора, естественно, должен выбрать синхронизирующие токены исходя из особенностей исходного языка. Хотя такая коррекция часто пропускает значительное количество символов без проверки на наличие ошибок, она достаточно проста и, в отличие от некоторых методов, гарантирует отсутствие закливания. В случаях, когда несколько ошибок в одной инструкции встречаются очень редко, этот метод работает вполне адекватно.

Уровень фразы. При обнаружении ошибки синтаксический анализатор может выполнить локальную коррекцию оставшегося входного потока. Т.о., он может заменить префикс остальной части потока некоторой строкой, которая позволит синтаксическому анализатору продолжить работу. Типичная локальная коррекция может состоять в удалении лишней точки с запятой или вставке недостающей. Выбор способа локальной коррекции – прерогатива разработчика компилятора. Естественно, нужно быть предельно осторожным при выборе способа коррекции, чтобы он не привёл, например, к бесконечному циклу.

Этот тип замещения может скорректировать любую входную строку и используется в ряде компиляторов, исправляющих ошибки. Основной его недостаток заключается в сложности обработки ситуации, когда реально ошибка располагается до точки обнаружения.

Произведения ошибок. Если известны наиболее распространённые ошибки, можно расширить грамматику языка произведениями, порождающими ошибочные конструкции. Затем строится синтаксический анализатор на основе такой расширенной грамматики. Если синтаксический анализатор использует одну из «ошибочных» произведений, можно сгенерировать корректное диагностическое сообщение для распознанной синтаксическим анализатором ошибки.

Нерекурсивный предиктивный анализ

Является одним из вариантов синтаксического анализа. Может быть построен с помощью явного использования стека. Ключевая проблема предиктивного анализа заключается в определении продукции, которую следует применить к нетерминалу. Нерекурсивный синтаксический анализатор, представленный на рис. 1, ищет необходимую для анализа продукцию в таблице разбора (далее будет показано, как строится эта таблица на основе грамматики).

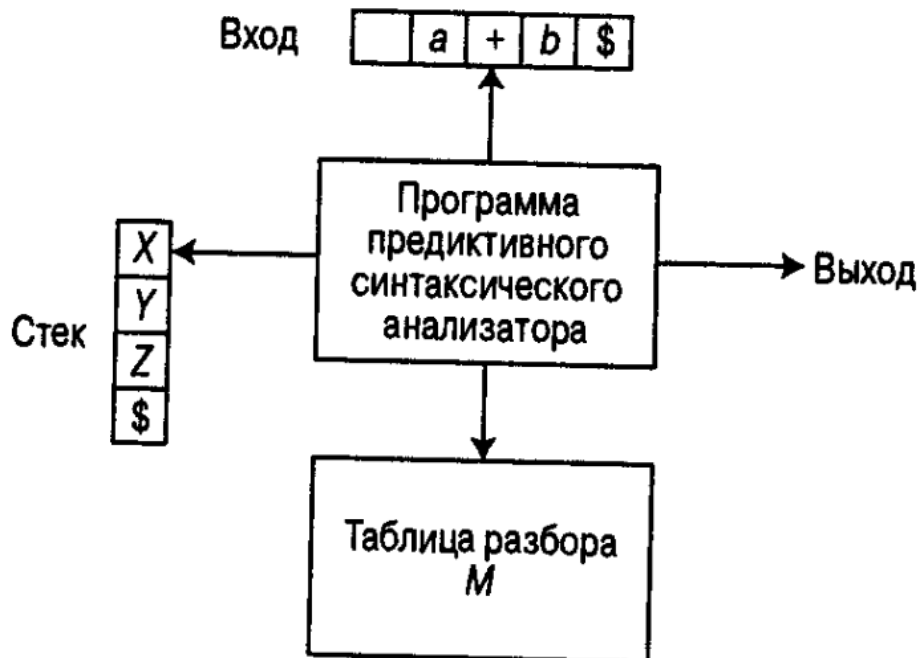


Рис 1. Модель нерекурсивного предиктивного синтаксического анализатора

Для изложения дальнейшего материала введём некоторые простейшие соглашения (они действуют по умолчанию, если не сказано иного):

1. Прописными буквами латинского алфавита будут обозначаться нетерминалы;
2. Строчными буквами латинского алфавита – терминалы;
3. Символ \$ обозначает дно стека, конец строки и т.п.
4. Символ $\Rightarrow$ обозначает пустой символ.

Предиктивный синтаксический анализатор, управляемый таблицей, имеет входной буфер, стек, таблицу разбора и выходной поток. Входной буфер содержит анализируемую строку с маркером её правого конца – специальным символом. Стек содержит последовательность символов грамматики с \$ на конце. Изначально стек содержит стартовый символ грамматики непосредственно над символом \$. Таблица разбора представляет собой двухмерный массив $M[A, a]$, где A – нетерминал, а a – терминал или символ \$.

Синтаксический анализатор управляется программой, которая работает следующим образом. Программа рассматривает X – символ на вершине стека, и a – текущий входной символ. Эти 2 символа определяют действия синтаксического анализатора. Имеется 3 варианта:

- ♦ Если $X = a = \$$, синтаксический анализатор прекращает работу и сообщает об успешном завершении разбора;
- ♦ Если $X = a \neq \$$, синтаксический анализатор снимает со стека X и перемещает указатель входного потока к следующему символу;

- Если X представляет собой нетерминал, программа рассматривает запись $M[X,a]$ из таблицы разбора M . Эта запись представляет собой либо X -продукцию грамматики, либо запись об ошибке. Если, например, $M[X,a] = \{X \cdot UVW\}$, синтаксический анализатор замещает X на вершине стека на WVU (с U на вершине стека). В данном случае полагается, что в качестве выхода синтаксический анализатор просто выводит использованную продукцию, но, конечно же, здесь может выполняться любой необходимый код. Если $M[X,a] = \text{error}$, синтаксический анализатор вызывает программу восстановления после ошибки.

Алгоритм: нерекурсивный предиктивный анализ

Вход: строка w и таблица разбора M грамматики G .

Выход: в случае $w \in L(G)$ – левое порождение w ; в противном случае – сообщение об ошибке.

Метод: Изначально синтаксический анализатор находится в конфигурации с $\$S$ (на его вершине – стартовый символ S грамматики G) и строкой $w\$$ во входном буфере. Программа, использующая для анализа входного потока таблицу предиктивного разбора M , приведена ниже:

Установить указатель ip на первый символ $w\$$;

repeat

обозначим через X символ на вершине стека, а через a – символ, на который указывает ip .

if X – терминал или $\$$ **then**

if $X = a$ **then**

снять со стека X и переместить ip

else error()

else /* X - нетерминал */

if $M[X, a] = X \cdot Y_1 Y_2 \dots Y_k$ **then begin**

снять X со стека;

Поместить в стек $Y_1, Y_2, \dots, Y_k$ с Y_1 на вершине стека;

Вывести продукцию $X \cdot Y_1 Y_2 \dots Y_k$

end

else error()

until $X = \$$ /* Стек пуст */

Пример 1. Рассмотрим грамматику со следующими productions

$E \rightarrow TA$

$A \rightarrow +TA \mid \$$

$T \rightarrow FB$

$B \rightarrow *FB \mid \$$

$F \rightarrow (E) \mid id$

Таблица предиктивного анализа этой грамматики показана на рис 2. Пустые ячейки означают ошибки; непустые указывают продукции, с помощью которых заменяются нетерминалы на вершине стека.

Нетерминал	Входной символ					
	id	+	*	(	)	\$
E	E • TA			E • TA		
A		A • +TA			A • ⇨	A • ⇨
T	T • FB			T • FB		
B		B • ⇨	B • *FB		B • ⇨	B • ⇨
F	F • id			F • (E)		

Рис 2. Таблица разбора

При входном потоке **id+id*id** предиктивный синтаксический анализатор осуществляет последовательность перемещений, показанную на рис. 3. Входной указатель при перемещении указывает на крайний слева символ в столбце «Вход».

Стек	Вход	Выход
\$E	id+id*id \$	
\$AT	id+id*id \$	E • TA
\$ABF	id+id*id \$	T • FB
\$ABid	id+id*id \$	F • id
\$AB	+id*id \$	
\$A	+id*id \$	B • ⇨
\$AT+	+id*id \$	A • +TA
\$AT	id*id \$	
\$ABF	id*id \$	T • FB
\$ABid	id*id \$	F • id
\$AB	*id \$	
\$ABF*	*id \$	B • *FB
\$ABF	id \$	
\$ABid	id \$	F • id
\$AB	\$	
\$A	\$	B • ⇨
\$	\$	A • ⇨

Рис 3. Перемещения предиктивного синтаксического анализатора

FIRST и FOLLOW

При построении предиктивного синтаксического анализатора весьма полезными оказываются две функции, связанные с грамматикой G, - FIRST и FOLLOW, которые обеспечивают заполнение таблицы предиктивного анализа грамматики G. Множества токенов, порождаемые функцией FOLLOW, могут также использоваться как синхронизирующие токены в процессе восстановления после ошибки в «режиме паники».

Если $\Rightarrow$ – произвольная строка символов грамматики, то определим $FIRST(\Rightarrow)$ как множество терминалов, с которых начинаются строки, выводимые из $\Rightarrow$. Если $\Rightarrow \xrightarrow{\alpha} \Rightarrow'$, то $\Rightarrow \in FIRST(\Rightarrow')$

Определим $FOLLOW(A)$ для нетерминала A как множество терминалов **a**, которые могут располагаться непосредственно справа от A в некоторой сентенциальной форме, т.е. множество терминалов **a**, таких, что существует порождение вида $S \xrightarrow{\alpha} Aa\beta$ для некоторых α и β . В процессе приведения между A и **a** могут появиться символы, но они порождают $\Rightarrow$ и в конечном счёте исчезают.

Если A может оказаться крайним справа символом некоторой сентенциальной формы, то $\$ \uparrow \text{FOLLOW}(A)$.

Чтобы вычислить $\text{FIRST}(X)$ для всех символов грамматики X , применяются следующие правила до тех пор, пока ни к одному из множеств FIRST не смогут быть добавлены ни терминалы, ни $\Rightarrow$.

- Если X – терминал, то $\text{FIRST}(X) = \{X\}$
- Если имеется продукция $X \Rightarrow$, добавим $\Rightarrow$ к $\text{FIRST}(X)$
- Если X – нетерминал и имеется продукция $X \Rightarrow Y_1 Y_2 \dots Y_k$, то поместим a в $\text{FIRST}(X)$, если для некоторого i $a \uparrow \text{FIRST}(Y_i)$ и $\Rightarrow$ входит во все множества $\text{FIRST}(Y_1), \dots, \text{FIRST}(Y_{i-1})$, т.е. $Y_1 \dots Y_{i-1} \nearrow \Rightarrow$. Если $\Rightarrow$ имеется во всех $\text{FIRST}(Y_i), i=1..k$, то добавляем $\Rightarrow$ к $\text{FIRST}(X)$. Например, всё, что находится в множестве $\text{FIRST}(Y_1)$, есть и в множестве $\text{FIRST}(X)$. Если Y_1 не порождает $\Rightarrow$, то больше ничего не добавляется к $\text{FIRST}(X)$, но если $Y_1 \nearrow \Rightarrow$, то к $\text{FIRST}(X)$ добавляется $\text{FIRST}(Y_2)$ и т.д.

Теперь для любой строки $X_1 X_2 \dots X_n$ вычислить FIRST можно следующим образом: добавим к $\text{FIRST}(X_1 \dots X_n)$ все не- $\Rightarrow$ символы из $\text{FIRST}(X_1)$. Добавим также все не- $\Rightarrow$ символы из $\text{FIRST}(X_2)$, если $\Rightarrow \uparrow \text{FIRST}(X_1)$, все не- $\Rightarrow$ символы из $\text{FIRST}(X_3)$, если $\Rightarrow$ имеется как в $\text{FIRST}(X_1)$, так и в $\text{FIRST}(X_2)$ и т.д. И, наконец, добавим $\Rightarrow$ к $\text{FIRST}(X_1 X_2 \dots X_n)$ если для всех i $\text{FIRST}(X_i)$ содержит $\Rightarrow$.

Чтобы вычислить $\text{FOLLOW}(A)$ для всех нетерминалов A , будем применять следующие правила до тех пор, пока ни к одному множеству FOLLOW нельзя будет добавить ни одного символа.

- Поместим $\$$ в $\text{FOLLOW}(S)$, где S – стартовый символ, а $\$$ – правый ограничитель входного потока.
- Если имеется продукция $A \Rightarrow B \Leftarrow$, то все элементы множества $\text{FIRST}(\Leftarrow)$, кроме $\Rightarrow$, помещаются в множество $\text{FOLLOW}(B)$.
- Если имеется продукция $A \Rightarrow B$ или $A \Rightarrow B \Leftarrow$, где $\text{FIRST}(\Leftarrow)$ содержит $\Rightarrow$ (т.е. $\Leftarrow \nearrow \Rightarrow$), то все элементы из множества $\text{FOLLOW}(A)$ помещаются в множество $\text{FOLLOW}(B)$.

Пример 2. Обратимся вновь к грамматике из примера 1

Для неё

$\text{FIRST}(E) = \text{FIRST}(T) = \text{FIRST}(F) = \{ (, \text{id} \}$

$\text{FIRST}(A) = \{ +, \Rightarrow \}$

$\text{FIRST}(B) = \{ *, \Rightarrow \}$

$\text{FOLLOW}(E) = \text{FOLLOW}(A) = \{), \$ \}$

$\text{FOLLOW}(T) = \text{FOLLOW}(B) = \{ +,), \$ \}$

$\text{FOLLOW}(F) = \{ +, *,), \$ \}$

Построение таблиц предиктивного анализа

Идея, лежащая в основе алгоритма построения таблицы, состоит в следующем. Предположим, что $A \Rightarrow$ представляет собой продукцию, у которой $a \uparrow \text{FIRST}(\Rightarrow)$. Тогда синтаксический анализатор заменит A строкой $\Rightarrow$ при текущем входном символе a . Единственная сложность возникает при $\Rightarrow = \Rightarrow$ или $\Rightarrow \nearrow \Rightarrow$. В этом случае мы снова должны заменить A на $\Rightarrow$, если текущий входной символ имеется в $\text{FOLLOW}(A)$ или из входного потока получен $\$$, который входит в $\text{FOLLOW}(A)$.

Алгоритм

Вход: грамматика G

Выход: таблица анализа M .

Метод:

1. Для каждой продукции грамматики $A \rightarrow \alpha$ выполнять шаги 2 и 3
2. Для каждого терминала a из $FIRST(\Rightarrow)$ добавляем $A \rightarrow \alpha$ в ячейку $M[A, a]$
3. Если в $FIRST(\Rightarrow)$ входит $\Rightarrow$, для каждого терминала b из $FOLLOW(A)$ добавим $A \rightarrow \alpha$ в ячейку $M[A, b]$. Если $\Rightarrow$ входит в $FIRST(\Rightarrow)$, а $\$$ – в $FOLLOW(A)$, добавим $A \rightarrow \alpha$ в ячейку $M[A, \$]$
4. Сделать каждую неопределённую ячейку таблицы M указывающей на ошибку

Данный алгоритм может быть применён к любой грамматике G для построения таблицы разбора M . Однако для некоторых грамматик M в одной ячейке таблицы может оказаться несколько записей. Например, если G – неоднозначная или леворекурсивная грамматика. Как поступить в данном случае? Один выход состоит в преобразовании грамматики, устраняющем левую рекурсию, и левой факторизации, в надежде получить грамматику, в таблице анализа которой отсутствуют ячейки с несколькими записями (пытливый студент отправляется к дополнительной литературе за разъяснениями по этому поводу). К сожалению, имеются грамматики, никакие изменения которых не приведут к желаемому результату. Грамматика в задании к данной лабораторной работе лишена подобной неоднозначности (однако настоятельно рекомендую проверить – это займёт не так много времени, как кажется, зато избавит от многочасовых (и, скорее всего, безрезультатных) поисков ошибки в вашем коде в том случае, если я ошибся)

Восстановление после ошибок в предиктивном анализе

Синтаксический анализатор находит ошибку в тот момент, когда терминал на вершине стека не соответствует очередному входному символу или на вершине стека находится нетерминал A , очередной входной символ – a , а ячейка таблицы синтаксического анализа $M[A, a]$ пуста.

Восстановление после ошибки в «режиме паники» основано на пропуске символов из входного потока до тех пор, пока не будет обнаружен токен из выбранного множества синхронизирующих токенов. Эффективность этого метода зависит от выбора синхронизирующего множества. Множества должны выбираться так, чтобы синтаксический анализатор быстро восстанавливался после часто встречающихся ошибок. Вот некоторые эвристические правила.

- ♦ В качестве первого приближения можно поместить в синхронизирующее множество для нетерминала A все символы из множества $FOLLOW(A)$.
- ♦ Множества $FOLLOW(A)$ недостаточно. Например, если инструкция завершается точкой с запятой, то ключевое слово, с которого начинается инструкция, может не оказаться во множестве $FOLLOW$ нетерминалов, порождающих выражения. Отсутствие точки с запятой после присвоения может, таким образом, привести к пропуску ключевого слова, с которого начинается новая инструкция. Зачастую в языке имеется иерархическая структура конструкций – например, выражения появляются в инструкциях, которые находятся в блоках и т.д. В таком случае к синхронизирующему множеству конструкции нижнего уровня можно добавить символы, с которых начинаются конструкции более высокого уровня. Например, можно добавить ключевые слова, с которых начинаются инструкции, в синхронизирующие множества нетерминалов, порождающих выражения.

- ♦ Если добавить символы из $FIRST(A)$ в синхронизирующее множество для нетерминала A , станет возможным продолжение анализа в соответствии с A , когда во входном потоке появится символ из множества $FIRST(A)$.
- ♦ Если нетерминал может порождать пустую строку, то в качестве продукции по умолчанию может использоваться продукция, порождающая $\Rightarrow$. Это может отсрочить обнаружение ошибки, зато не может вызвать её пропуск и сокращает число нетерминалов, которые должны быть рассмотрены в процессе восстановления после ошибки.
- ♦ Если терминал на вершине стека не может быть сопоставлен с входным символом, то простейший метод состоит в снятии терминала со стека, генерации сообщения о вставке терминала в программу и продолжении синтаксического анализа. По сути, при этом подходе синхронизирующее множество токена состоит из всех остальных токенов.

Пример 3

Использование символов из множеств FOLLOW и FIRST в качестве синхронизирующих достаточно неплохо работает при разборе грамматики из примера 1. Таблица синтаксического анализа для этой грамматики показана ниже. “Synch” указывает синхронизирующие токены, полученные из множества FOLLOW соответствующего терминала. Множества FOLLOW для нетерминалов получены из примера 2.

Нетерминал	Входной символ					
	id	+	*	(	)	\$
E	E • TA			E • TA	Synch	Synch
A		A • +TA			A • $\Rightarrow$	A • $\Rightarrow$
T	T • FB	Synch		T • FB	Synch	Synch
B		B • $\Rightarrow$	B • *FB		B • $\Rightarrow$	B • $\Rightarrow$
F	F • id	Synch	Synch	F • (E)	Synch	Synch

Таблица используется следующим образом. Если синтаксический анализатор просматривает ячейку $M[A,a]$ и обнаруживает, что она пуста, то входной символ a пропускается. Если в этой ячейке находится запись Synch, то нетерминал снимается с вершины стека в попытке продолжить синтаксический анализ. Если токен на вершине стека не соответствует входному символу, то он снимается со стека.

Пример 4. В случае ошибочного ввода $)id*+id$ синтаксический анализатор и механизм восстановления после ошибок поведут себя так:

Стек	Вход	Примечание
\$E	)id*+id\$	Ошибка, пропускаем «)»
\$E	id*+id\$	id $\uparrow$ FIRST(E)
\$AT	id*+id\$	
\$ABF	id*+id\$	
\$ABid	id*+id\$	
\$AB	*+id\$	
\$ABF*	*+id\$	
\$ABF	+id\$	Ошибка, $M[F,+]$ = Synch
\$AB	+id\$	F снимается со стека
\$A	+id\$	
\$AT+	+id\$	

\$AT	id\$	
\$ABF	id\$	
\$ABid	id\$	
\$AB	\$	
\$A	\$	
\$	\$	

Приведённое обсуждение метода восстановления после ошибки в «режиме паники» не касалось важного вопроса сообщений об ошибках. Вообще говоря, разработчик компилятора должен обеспечить вывод информативных сообщений об обнаруженных ошибках.

Восстановление на уровне фразы. Реализуется заполнением пустых ячеек в таблице предиктивного синтаксического анализа указателями на подпрограммы обработки ошибок. Эти подпрограммы могут изменять, вставлять или удалять символы входного потока и выводить соответствующие сообщения об ошибках. Кроме того, они могут снимать элементы со стека. При таком способе восстановления возникает вопрос, можно ли разрешить изменение символов в стеке или только размещение в стеке новых символов, поскольку после этого шаги, выполняемые синтаксическим анализатором, могут не соответствовать порождению ни одного слова языка вообще. В любом случае нужно гарантировать, что программа не заиклится. Надёжная защита от заикливания – проверка того, что каждое действие по восстановлению после ошибки в конечном счёте приводит к обработке очередного входного символа (или к снятию элементов со стека, если достигнут конец входного потока).

Задание на лабораторную работу

Написать синтаксический анализатор, обнаруживающий наибольшее число ошибок, для приведённой ниже грамматики (данная грамматика является упрощённым вариантом грамматики языка C):

```

<program>: <type> 'main' '(' ')' '{' <statement> '}'
<type>: 'int'
      | 'bool'
      | 'void'
<statement>:
      | <declaration> ';'
      | '{' <statement> '}'
      | <for> <statement>
      | <if> <statement>
      | <return>
<declaration>: <type> <identifier> <assign>
<identifier>: <character><id_end>
<character>: 'a' | 'b' | 'c' | 'd' | 'e' | 'f' | 'g' | 'h' | 'i' | 'j' | 'k' | 'l' | 'm' | 'n' | 'o' | 'p' | 'q' |
            'r' | 's' | 't' | 'u' | 'v' | 'w' | 'x' | 'y' | 'z' | 'A' | 'B' | 'C' | 'D' | 'E' | 'F' | 'G' |
            'H' | 'I' | 'J' | 'K' | 'L' | 'M' | 'N' | 'O' | 'P' | 'Q' | 'R' | 'S' | 'T' | 'U' | 'V' |
            'W' | 'X' | 'Y' | 'Z' | '_'
<id_end>:
      | <character><id_end>
<assign>:

```

```

    | '=' <assign_end>
<assign_end>: <identifier>
    | <number>
<number>: <digit><number_end>
<digit>: '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
<number_end>:
    | <digit><number_end>
<for>: 'for' '(' <declaration> ';' <bool_expression> ';' ')'
<bool_expression>: <identifier> <relop> <identifier>
    | <number> <relop> <identifier>
<relop>: '<' | '>' | '==' | '!='
<if>: 'if' '(' <bool_expression> ')'
<return>: 'return' <number> ';'

```

Данная грамматика записана с помощью вариации РБНФ. Нетерминалы заключены в <>, терминал (или последовательность терминалов) – в ‘’. Символ | обозначает альтернативу (т.е. «или»). Запись <N1><N2> означает, конкатенацию нетерминалов, а запись <N1> <N2> – последовательную запись нетерминалов, разделённых пробельными символами (пробельные символы аналогичны таковым в грамматике языка C). Запись вида <N1>: | <N2> означает, что нетерминал N1 можно заменить либо нетерминалом N2, либо пустой строкой. Стартовым нетерминалом грамматики является <program>. Назовём язык, описываемый данной граматикой, C-light.

Входные данные: файл с программой на языке C-light

Выходные данные:

1. **Задание минимум:** количество обнаруженных ошибок и сообщения о месте их возникновения
2. **Задание максимум:** количество обнаруженных ошибок, сообщения о месте их возникновения и о их типе, файл с таблицей, аналогичной таблице из примера 4.

Восстановление после ошибок реализовать в «режиме паники»

В качестве **дополнения** может быть реализован механизм восстановления после ошибок в режиме фразы, а также сгенерирован файл с таблицей предиктивного анализа.

Литература

При подготовке данной лабораторной работы использовалась книга Альфред Ахо, Рави Сети, Джеффри Ульман – Компиляторы. Принципы, технологии, инструменты – Вильямс – Москва Санкт-Петербург Киев 2003

Лабораторная работа №5

Разработка интерпретатора.

Теоретическая часть

Интерпретаторы важны по нескольким веским причинам. Во-первых, они могут обеспечивать удобную интерактивную среду (что зачастую удобнее, чем просто компилятор языка). Во-вторых, интерпретаторы языка обеспечивают превосходные интерактивные отладочные возможности. Интерпретатор позволяет, например, динамично устанавливать значения переменных и условий. В-третьих, большинство языков-запросов к базам данных работают в режиме интерпретации.

В данной работе будут даны общие методы и подходы к реализации интерпретатора на примере языка, который условно назовём CBAS (описание структуры программы на этом языке будет дано ниже). Язык представляет собой смесь языков BASIC и C, и упрощён почти до предела.

При разработке интерпретатора следует учесть тот факт, что каждая лексема имеет два формата: внешний и внутренний. Внешний формат – это строка символов, с помощью которой вы пишете программы на каком-либо языке программирования. Например, «while» – это внешняя форма ключевого слова while языка C. Можно построить интерпретатор из расчёта, что каждая лексема используется во внешнем формате, но это типичное решение проблемы программистом-непрофессионалом. Студентам четвёртого курса следует ориентироваться на внутренний формат лексемы, который является просто числом, и разрабатывать интерпретатор исходя из этой профессиональной точки зрения на проблему. Этот подход заключается в том, что каждой лексеме присваивается внутренний номер. Например, ключевое слово while может иметь порядковый номер 1, if – 2 и т.д. Преимущество внутреннего формата заключается в том, что программы, обрабатывающие числа, быстрее программ, обрабатывающих строки. Для реализации такого подхода необходима функция, которая берет из входного потока данных очередную лексему и преобразует ее из внешнего формата во внутренний.

Очень важно знать, какой тип лексемы возвращён. Например, анализатору выражений необходимо знать, является ли следующая лексема числом, оператором или переменной. Значение типа лексемы для процесса анализа в целом станет очевидным, когда вы приступите непосредственно к разработке интерпретатора.

Некоторые функции интерпретатора нуждаются в повторном просмотре лексемы. В этом случае лексема должна быть возвращена во входной поток. Например, это необходимо, когда очередная лексема является началом выражения. Тогда она помещается обратно во входной поток и вызывается функция обработки и вычисления этого выражения.

Спецификация языка CBAS

<character>: 'a' 'b' 'c' 'd' 'e' 'f' 'g' 'h' 'i' 'j' 'k' 'l' 'm' 'n' 'o' 'p' 'q' 'r' 's' 't' 'u' 'v' 'w' 'x' 'y' 'z' 'A' 'B' 'C' 'D' 'E' 'F' 'G' 'H' 'I' 'J' 'K' 'L' 'M' 'N' 'O' 'P' 'Q' 'R' 'S' 'T' 'U' 'V' 'W' 'X' 'Y' 'Z' '_'
<id_end>:

```

| <character><id_end>
<identifier>: <character><id_end>
<digit>: '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
<number_end>:
| <digit><number_end>
<number>: <digit><number_end>
<expression>: <term> '+' <expression>
| <term> '-' <expression>
| <term>
<term>: <factor> '*' <term>
| <factor> '/' <term>
| <factor>
<factor>: <identifier> | <number> | (<expression>)
<relop>: '<' | '>' | '=' | '!='
<bool_expression>: <expression> <relop> <expression>
<assign>: <identifier> '=' <expression>
<string>: '"' <любое количество отличных от '"' символов> '"'
<print>: 'print' <print_end> ';'
<print_end>: <string>
| <expression>
| <string> ',' <print_end>
| <expression> ',' <print_end>
<scan>: 'scan' <identifier> ';'
<for>: 'for' <identifier> '=' <expression> 'to' <expression>
<if>: 'if' <bool_expression>
<else>:
| 'else' '{' <statement> '}'
<statement>:
| <print> <statement>
| <scan> <statement>
| <assign> <statement>
| <for> '{' <statement> '}'
| <if> '{' <statement> '}' <else>
<program>: <statement>

```

Все переменные состоят только из букв и символа подчёркивания и являются глобальными. Такая конструкция, как объявление переменных, в языке CBAS не предусмотрена. Все переменные имеют тип данных, аналогичный `int`. Транслятор при транслировании исходной программы строит таблицу переменных. Таблица представляет собой пары «имя-значение». При обнаружении идентификатора транслятор просматривает таблицу в поисках такого же имени. Если его там нет, то создаётся новое вхождение в таблицу, и переменной присваивается какое-то начальное значение (например, 0).

Оператор **print** служит для вывода информации на экран. Он может печатать как символьные строки, так и значения выражений.

Оператор **scan** служит для считывания с клавиатуры значений в переменную. Выполнение программы приостанавливается, пока пользователь не введёт какое-либо число (т.к. все переменные имеют тип данных `int`).

Цикл **for** работает следующим образом. Переменной в цикле присваивается начальное значение, которое инкрементируется на единицу при каждой итерации. Выход из цикла происходит, когда переменная цикла БОЛЬШЕ ИЛИ РАВНА значению верхней границы цикла.

Некоторые особенности построения интерпретаторов.

Как уже упоминалось, интерпретатор оперирует внутренним представлением для лексем. Для преобразования во внутренний формат используется таблица

```
struct commands { /* Вспомогательная структура ключевых слов анализатора */
    std::string command;
    int token;
} table[];
```

Основной цикл работы интерпретатора состоит в пошаговом выполнении всех инструкций исходной программы. Все интерпретаторы выполняют операции путем считывания лексемы программы и выбора необходимой функции для ее выполнения. Например, если очередная лексема – это ключевое слово 'if', то запускается функция process_if(). В качестве параметра функция принимает указатель на обрабатываемую часть программы. Это может быть номер строки и символа в файле, какая-либо другая информация. Т.к. язык CBAS подразумевает вложенность конструкций, то каждая функция обработки той или иной лексемы вместе с выполнением специфических для данной лексемы функций должна вызывать общую функцию обработки. Пусть, например, программа состоит исключительно из циклов for и условий if. Тогда интерпретатор условно можно представить в виде следующих функций:

```
void process_if(char**);
void process_for(char**);

void main_loop(char* programm){
    int token = get_next_lexem(&program);
    switch (token){
        case IF:
            process_if(&program);
            break;
        case FOR:
            process_for(&program);
            break;
        default:
            //ошибка
    }
}

void process_if(char** pointer){
    //специфичные действия для if
    main_loop(*pointer);
}

void process_for(char** pointer){
```

```
//специфичные действия для for
main_loop(*pointer);
}
```

Циклы

В языке CBAS, точно также как и в C, допускается вложенность цикла for. Основной изюминкой при реализации этого оператора является сохранение информации о каждом вложенном цикле со ссылкой на внешний цикл. Для реализации этого используется стековая структура, которая работает следующим образом: начало цикла, информация о значении управляющей переменной цикла и ее конечном значении, а также место расположения цикла в теле программы заносятся в стек.

Каждый раз, при достижении '}', соответствующей концу цикла for, из стека извлекается информация о значении управляющей переменной, затем ее значение пересчитывается и сравнивается с конечным значением цикла. Если значение управляющей переменной цикла достигло своего конечного значения, выполнение цикла прекращается и выполняется оператор программы, следующий за циклом. В противном случае, в стек заносится новая информация, и выполнение цикла начинается с его начала. Таким же образом обеспечивается интерпретация и выполнение вложенных циклов. В стекоподобной структуре вложенных циклов каждый for должен быть закрыт соответствующей '}'.

Для реализации оператора цикла for стек должен иметь следующую структуру:

```
struct for_stack {
    int var; /* счетчик цикла */
    int target; /* конечное значение */
    char* location;
} fstack[FOR_COUNT]; /* стек для цикла for */
int ftop; /* индекс начала стека FOR */
```

Константа FOR_COUNT ограничивает уровень вложенности цикла. Переменная ftop всегда имеет значение индекса начала стека.

Вместо явного использования стековой структуры, можно воспользоваться механизмом локальных переменных языка, на котором реализуется интерпретатор. В этом случае вложенные операторы обрабатываются с помощью рекурсивного вызова соответствующих процедур, и необходимая информация сохраняется в локальных переменных вызывающей процедуры.

Порядок построения выражений

Имеется много вариантов анализа и вычисления выражений. Для использования полного синтаксического анализатора рекурсивного спуска мы должны представить выражение в виде рекурсивной структуры данных. Это означает, что выражение определяется в терминах самого себя. Если выражение можно определить с использованием только символов "+", "-", "*", "/" и скобок, то все выражения могут быть определены с использованием следующих правил:

Выражение => Терм [+Терм][-Терм]
Терм => Фактор [*Фактор]/[Фактор]
Фактор => Переменная, Число или (Выражение)

Очевидно, что некоторые части в выражении могут отсутствовать вообще. Квадратные скобки означают именно такие необязательные элементы выражения. Символ '=>' имеет смысл "производит".

Фактически, выше перечислены правила, которые обычно называют правилами вывода выражения. В соответствии с этими правилами терм можно определить так: "Терм является произведением или отношением факторов".

Приоритет операторов безусловен в описанных выражениях, то есть вложенные элементы включают операторы с более высоким приоритетом.

В связи с этим рассмотрим ряд примеров. Выражение

$$10+5*B$$

содержит два термина: "10" и "5*B". Они, в свою очередь, состоят из трех факторов: "10", "5" и "B", содержащих два числа и одну переменную. В другом случае выражение

$$14*(7-C)$$

содержит два фактора "14" и "(7-C)", которые, в свою очередь, состоят из числа и выражения в скобках. Выражение в скобках вычисляется как разность числа и переменной.

Можно преобразовать правила вывода выражений в множество общих рекурсивных функций, что и является зачастую основной формой синтаксического анализатора рекурсивного спуска. На каждом шаге анализатор такого типа выполняет специфические операции в соответствии с установленными алгебраическими правилами. Работу этого процесса можно рассмотреть на примере анализа выражения и выполнения арифметических операций. Пусть на вход анализатора поступает следующее выражение:

$$9/3-(100+56)$$

Анализатор в этом случае будет работать по такой схеме:

1. Берем первый терм: "9/3".
2. Берем каждый фактор и выполняем деление чисел, получаем результат "3".
3. Берем второй терм: "(100+56)". В этой точке стартует рекурсивный анализ второго выражения.
4. Берем каждый фактор и суммируем их между собой, получаем результат 156
5. Берем число, вернувшееся из рекурсии, и вычитаем его из первого: 3-156. Получаем итоговый результат "-153".

Вы должны помнить две основные идеи рекурсивного разбора выражений:

1. приоритет операторов является безусловным в продукционных правилах и определен в них;
2. этот метод синтаксического анализа и вычисления выражений очень похож на тот, который вы сами используете для выполнения таких же операций.

Задание на лабораторную работу

Создать интерпретатор языка CBAS.

Входные данные: исходный текст программы на языке CBAS.

Выходные данные: консоль с результатами ввода-вывода исходной программы, либо сообщения об ошибках, присутствующих в программе.

Лабораторная работа №6

Разработка процессора языка разметки документа.

Теоретическая часть

Язык разметки в компьютерной терминологии – набор символов или последовательностей, вставляемых в текст для передачи информации о его выводе или строении. Языки разметки используются везде, где требуется вывод форматированного текста: в типографии (SGML, TeX, PS, PDF), пользовательских интерфейсах компьютеров (troff, Microsoft Word, OpenOffice), Всемирной сети (HTML, XML).

Например, TeX – система компьютерной вёрстки, разработанная американским почётным профессором Дональдом Кнудом в целях создания компьютерной типографии. В неё входят средства для секционирования документов, для работы с перекрёстными ссылками и для набора сложных математических формул. Документы набираются на собственном языке разметки в виде обычных ASCII-файлов. Эти файлы транслируются специальной программой в файлы, «независимые от устройства», которые могут быть отображены на экране или напечатаны. Ядро TeX'a представляет собой язык низкоуровневой разметки, содержащий команды отступа и смены шрифта.

Каждый документ имеет три составляющие – содержание (смысловое наполнение), структуру и внешнее представление. Структура документа позволяет правильно определить составляющие его части и взаимоотношения между ними. Внешнее представление направлено на повышение эффективности восприятия информации читателем, что достигается за счет выделения смысловых частей документа теми или иными средствами, доступными для данной формы представления.

О разметке

В документе, помимо смыслового наполнения, должна содержаться некоторая метайнформация, позволяющая определить его структуру и внешнее представление. Такая метайнформация называется разметкой документа.

Разметка документа преследует следующие две основные цели:

- выделение смысловых частей (логических элементов) документа и связей между ними;
- указание действий, которые должны быть осуществлены с этими элементами.

Для достижения первой цели предназначена *структурная разметка*. Действия, направленные на получение внешнего представления, задаются *разметкой представления*.

В качестве примеров приведены два возможных способа разметки.

Пример 1

```
<div1 type="Section">
<head>Введение</head>
<p>Так уж сложилось, что большую часть информации человек
предпочитает хранить в виде документов...</p>
</div1>
```

Пример 2

```
<font face="Arial Bold" size=16>1. Введение<hspace size=20>  
<tab size=5><font face="Times New Roman" size=12>  
Так уж сложилось, что большую часть информации человек  
предпочитает хранить в виде документов...
```

В первом случае описан раздел, который имеет заголовок и текст в виде абзаца, то есть определяет структуру документа. Структурная разметка говорит о том, как текст устроен, то есть из каких он частей состоит, и как эти части друг с другом соотносятся.

Во втором случае показано, каким образом данный текст должен быть отображен на бумаге или на мониторе — выделить шрифтом Arial Bold размера 16, отступить по вертикали 20, сделать табуляцию 5, выделить шрифтом Times New Roman размера 12. Здесь мы имеем дело с разметкой представления документа, которая говорит о том, что делать с текстом, как его отображать.

Исторически разметка представления появилась раньше, и в течение длительного времени разметка документа была ориентирована исключительно на внешний (бумажный) вид документа. Но в последнее время ситуация существенно меняется — быстрый рост числа документов, их создание, хранение и использование в электронном виде, автоматизированная обработка и обмен документами предъявляют новые требования к разметке. В числе этих требований — независимость от среды представления, возможность осуществления эффективного поиска, возможность повторного использования как документа целиком, так и отдельных его элементов.

Быстрый рост количества документов привел к тому, что поиск нужной информации стал занимать все больше и больше времени. Например, если необходимо найти в Интернет информацию об авторе статей, то простой контекстный поиск даст огромное количество ссылок на те места, где встречается фамилия автора. После чего придется либо просмотреть все полученные ссылки, либо задавать дополнительную информацию для сужения области поиска. Если бы мы могли сразу указать, что фамилию следует искать только среди авторов журнальных статей технического плана, это во много раз упростило бы поиск. Но для этого необходимо, чтобы документы, среди которых ведется поиск, были размечены должным образом с явным выделением элементов "автор", "тематика" и т.п.

Возможность повторного использования документов или отдельных его частей приводит к тому, что мы не составляем каждый раз заново отчет или деловое письмо, используем в своей работе шаблоны контрактов, изменяя лишь некоторую существенную для данного случая информацию. Но делаем мы это преимущественно вручную. Если говорить об автоматизированном формировании, связывании, повторном использовании документов, то это становится возможным только тогда, когда документы как информационные объекты являются структурированными, а используемая метainформация полно и ясно описывает характеристики каждого элемента документа.

Все перечисленные задачи можно решить, используя исключительно структурный подход при разметке документов. Именно структурная разметка позволяет выделять смысловые элементы, определять их связи с другими элементами как в рамках одного документа, так и вне этих рамок.

Характерной особенностью многих языков разметки является блочная структура всего документа. Концепция следующая: документ – это блок, который состоит из блоков, которые в свою очередь состоят из блоков и т.д.

Блок		Блок		Блок
		Блок	Блок	
Блок	Блок		Блок	
	Блок		Блок	
Блок	Блок	Блок		

Разметка текста осуществляется с помощью специальных управляющих кодов, присутствующих в документе. Одним из вариантов таких кодов являются **тэги (tag)**. Тэги представляют собой ключевые слова, описывающие структуру разметки. Кроме того, тэги могут иметь *атрибуты*, которые задают разметку представления. Например

<center color=red background=black> Текст красного цвета на чёрном фоне в центре страницы </center>

Тэги <center></center> образуют логические скобки, которые задают вывод заключённого между ними текста в центре. Тэг имеет атрибуты color и background, которые задают цвет текста и фона соответственно.

Вложенность блоков. Как уже упоминалось, языки разметки имеют блочную структуру. Предположим, что есть пара тэгов <block> </block>, задающих блок, и теги <row> </row> и <column> </column>, задающие строку и столбец в этих блоках. Тогда структура, представленная на рис. 1, может иметь следующий вид

A	B	E
C	D	
F		

Рис 1

```

<block>
  <column>
    <row>
      <block>
        <row>
          <column> A </column>
          <column> B </column>
        </row>
        <column> C </column>
        <column> D </column>
      </row>
    </row>
  </block>
</column>

  <row> F </row>
</column>

```

<column> E </column>
</block>

Спецификация языка разметки eMark

Данный язык разметки предназначен для оформления вывода текста на стандартную консоль, которая имеет 24 строки по 80 символов каждая. Соответственно страница выводимого текста может содержать максимум 24 строки и 80 столбцов. Поддерживаемые тэги и их атрибуты:

1. <block rows=число columns=число> </block>
атрибуты rows и columns задают число строк и столбцов в текущем блоке
2. <column valign=вертикальное_выравнивание
halign=горизонтальное_выравнивание textcolor=цвет bgcolor=цвет
width=число> </column>
вертикальное_выравнивание – одно из значений top, center, bottom
(выравнивание по верхнему краю, по центру и по нижнему краю соответственно)
горизонтальное_выравнивание – одно из значений left, center, right
(выравнивание по левому краю, по центру и по правому краю соответственно)
цвет – цифра от 0 до 15 включительно, соответствующая одному из цветов, в которые может быть раскрашена консоль
число – ширина столбца в символах
3. <row valign=вертикальное_выравнивание
halign=горизонтальное_выравнивание textcolor=цвет bgcolor=цвет
height=число> </row>
смысл атрибутов аналогичен одноимённым атрибутам тэга <column>.
Атрибут height задаёт высоту строки как количество строчек консоли
Дополнительные условия и ограничения:
 1. Документ должен состоять как минимум из одного тэга <block>
 2. Атрибуты rows и columns тэга <block> обязательны и не могут быть больше высоты (ширины) элемента, в который данный тэг вложен
 3. Количество вложенных тэгов <column> и <row> для каждого тэга <block> должно соответствовать значению атрибутов columns и rows
 4. Значения по умолчанию для тегов:
 - ♦ valign = top
 - ♦ halign = left
 - ♦ textcolor = 15 (белый)
 - ♦ bgcolor = 0 (чёрный)
 - ♦ width (только для последнего тэга <column> в тэге <block>, для остальных должно быть указано явно) равно значению ширины блока за вычетом суммы ширин всех предыдущих столбцов в блоке
 - ♦ height (только для последнего тэга <row> в тэге <block>, для остальных должно быть указано явно) равно значению высоты блока за вычетом суммы высот всех предыдущих строк в блоке
 - ♦ columns = 1 и rows = 0 только для внешнего тэга <block>, для остальных должно быть указано явно
 5. Каждому тэгу должен соответствовать закрывающий тэг

Пример простейшего документа на языке разметки eMark:

```
<block>  
    <column>"Hellow world" example using eMark</column>  
</block>
```

Задание на лабораторную работу

Написать процессор, обрабатывающий документы на языке eMark.

Входные данные: текстовый файл, содержащий документ на языке eMark.

Выходные данные: консоль с отформатированным текстом в соответствии с управляющими инструкциями, либо с сообщениями об ошибках в документе